Performance & Security Report

Zamzam Fashion — .NET 10 API (backend), pre-deploy

Prepared 7 November 2026 · repository `D:\New_Desktop\Zamzam Fashion`, branch `feature/day4-tailoring` (3332e47)

> **Scope and method.** Source review of the backend (`Program.cs`, security middleware, Identity/cookie setup, rate limits, upload pipeline, EF queries, config) plus a read-only look at the teammate's `frontend/next.config.ts`. Live checks against the API running locally (`http://localhost:5055`, Development) on the real Supabase dev database: response headers on 9 endpoints, 30-request latency samples, rate-limit trigger, IP-spoof, CSRF and CORS probes — passive only, no login attempts. Dependencies scanned with `dotnet list package --vulnerable --include-transitive` and `npm audit`. **Not measurable yet:** the production server (MonsterASP not deployed), IIS headers, TLS, the real latency from the production region, and the storefront's Core Web Vitals (frontend still on mock data). CHECK items list the confirming tool in Section 4. Section 6 is a **modelled calculation**, not a load test.

Status key: **OK** observed and sound · **CHECK** needs a measurement or access not available yet · **FIX** issue visible in the code or the live response.

Summary

The API is in good shape for its stage: strict security headers, a hardened owner cookie with a CSRF guard, working per-IP rate limits, a careful photo pipeline and no vulnerable packages. The **main security gap** is that the real customer IP never reaches the API through Vercel — the frontend rewrite can't send the proxy secret — so at launch every customer would share one rate limit and a busy hour would start returning 429s. The **main performance risks** are uncached product endpoints, very noisy Information-level logging on a shared host, and photo decoding that can use ~160 MB per upload. Everything else is configuration to finish during the Day 4C deploy.

Area	Finding	Status
Security headers	nosniff, DENY, no-referrer, CSP `default-src 'none'`, CORP, HSTS (non-dev); Server header off	**OK**
Auth & CSRF	HttpOnly + Secure + SameSite=Strict cookie, CSRF header enforced (live 400), lockout 5 / 15 min	**OK**
Rate limits	Per-IP limits work live (31st tracking call → 429, Retry-After 60); X-Forwarded-For spoof ignored	**OK**
Real client IP behind Vercel	Frontend rewrite never sends `X-Proxy-Secret` → every customer shares Vercel's IP and one limit	**FIX**
Logging volume	370 SQL commands + every MVC step logged at Information in a short test (4,246 lines)	**FIX**
Product caching	`/api/products` and `/api/products/{slug}` send no Cache-Control	**FIX**
Photo processing memory	Full-size decode up to 40 MP ≈ 160 MB RAM per upload; anonymous tailoring upload	**FIX**
Data-protection keys	Production falls back to DPAPI LocalMachine on a shared Windows host	**FIX**
Migrations on startup	`Database:MigrateOnStartup` defaults to true in production	**FIX**
Response compression	JSON is sent uncompressed (12 KB tailoring catalog)	**FIX**
Dependencies	NuGet: 0 vulnerable. npm (frontend): 8 moderate, all in `minimatch` dev tooling	**OK**
Production host	MonsterASP .NET 10 support, region, HTTPS redirect, IIS headers not yet verifiable	**CHECK**

1. Performance

Observation	Impact	Status
Hosting: API on MonsterASP (IIS, Windows), frontend on Vercel, DB + images on Supabase Frankfurt	Every DB round trip pays the server→Frankfurt distance	CHECK
Measured locally (Egypt laptop → Supabase Frankfurt, 30 requests each): categories p50 65 ms; product list p50 253 ms / p95 353 ms; product detail p50 224 ms; tailoring catalog p50 158 ms	List ≈ 4 DB round trips (split query + count) × ~60 ms. On an EU server this should drop a lot	CHECK
`Cache-Control: public,max-age=300` on categories, site settings, delivery zones, tailoring catalog	Vercel/browser serve repeats without touching the API	OK
No Cache-Control on `GET /api/products` and `/api/products/{slug}` (the hottest endpoints)	Every page view hits the DB; checkout re-checks stock anyway, so a short cache is safe	FIX
No response compression (no `Content-Encoding` with `Accept-Encoding: gzip, br`)	JSON compresses ~5–8×; small now, grows with the catalog	FIX
Serilog has no `MinimumLevel` override (it ignores `Logging:LogLevel`), so EF SQL and MVC internals log at Information	CPU + disk on shared hosting; real errors drown in noise	FIX
No `Max Pool Size` in the connection string; Supabase session pooler has a small client limit	Under load the API can open more connections than the pooler allows → timeouts	CHECK
`SkiaImageProcessor` decodes the full image (cap 40 MP) and the upload is buffered twice	One 40 MP photo ≈ 160 MB RAM; two at once can hit the app-pool memory cap	FIX
Paging capped (`Paging.Normalize`), `AsNoTracking` on reads, split queries for products	No unbounded lists or N+1 queries found	OK
Images: WebP 400/800/1600 generated once on upload, served from Supabase CDN, never from the API	Image bytes never cost API CPU or bandwidth	OK
Storefront LCP / INP / CLS	Frontend not connected yet (mock data) — measure on Day 5	CHECK

Recommendations

- **Cache the product endpoints for 60 s.** Stock is re-checked inside the checkout transaction, so a minute-old card is harmless; on Day 5 add Next.js ISR (`revalidate: 60`) on top.
- **Turn on response compression for JSON.** Two lines in `Program.cs`; check Vercel doesn't already re-compress before relying on it.
- **Quiet the logs.** Serilog reads its own `Serilog:MinimumLevel` section, not `Logging:LogLevel`. Keep the one request-summary line per call.
- **Pick an EU MonsterASP datacenter** so API → Supabase Frankfurt is a few ms, not ~60 ms per round trip.
- **Cap the pool:** `Max Pool Size=10;Connection Idle Lifetime=60` in the connection string, below Supabase's pooler limit.
- **Decode photos scaled.** Use `codec.GetScaledDimensions` / `SKCodec.GetPixels` with a sample size so a 6000 px photo decodes near 1600 px; read the upload into one buffer only.

```
// Program.cs
builder.Services.AddResponseCompression(o => { o.EnableForHttps = true; o.MimeTypes = ["application/json"]; });
app.UseResponseCompression();   // before MapControllers

// CatalogController — products list + detail (stock is re-checked at checkout)
[ResponseCache(Duration = 60, Location = ResponseCacheLocation.Any)]
```

Targets to measure against

Metric	Good	Measured now	Where to measure
API p95, uncached product list	≤ 300 ms	353 ms (laptop in Egypt → Frankfurt)	curl ×30 from the production server region
API p95, cached endpoints	≤ 100 ms	98 ms categories (uncached locally)	curl / k6
LCP	≤ 2.5 s	—	PageSpeed Insights, mobile (Day 5)
INP	≤ 200 ms	—	PageSpeed Insights / DevTools
CLS	≤ 0.1	—	PageSpeed Insights / Lighthouse
First-load weight	< 1.5 MB	API JSON part: 9.5 KB	DevTools Network, Slow 4G

2. Security

Observation	Risk	Status
Headers on every response incl. 401/404: `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `CSP default-src 'none'`, `CORP same-site`; HSTS outside Development	Correct for a JSON-only API	**OK**
Kestrel `Server` header off and removed; IIS may still add `Server` / `X-Powered-By: ASP.NET`	Version disclosure	**CHECK**
No `UseHttpsRedirection`; comment in `Program.cs` still says "Caddy/Cloudflare" (old plan)	If MonsterASP serves plain HTTP, HSTS alone doesn't help the first request	**CHECK**
`next.config.ts` rewrite cannot add headers, so `X-Proxy-Secret` never arrives; `ClientIpMiddleware` also stays silent when the secret is missing	Whole store shares one limit: 11th order in 10 min (any customer) → 429; 6 failed logins/min anywhere block the owner	**FIX**
Cookie `zz.auth`: HttpOnly, Secure, SameSite=Strict, 8 h sliding; CSRF header on admin/auth writes (live: 400 `MISSING_CSRF_HEADER`)	Session theft and CSRF covered	**OK**
All 10 admin controllers carry `[Authorize(Policy = Owner)]`; anonymous `/api/admin/orders` → 401 live	No open admin route. Add a fallback policy as a safety net	**OK**
Login: 5/min per IP + lockout 5 tries / 15 min; password ≥ 10 chars + common-password list	Brute force is slow	**OK**
Owner account still has `mustChangePassword = true` (smoke test)	Seed password still in use	**FIX**
Uploads: magic-number check, 5 MB cap + `RequestSizeLimit`, re-encode strips EXIF/GPS, private bucket + signed links for tailoring photos	Solid upload pipeline	**OK**
Production data-protection keys: certificate if configured, else DPAPI LocalMachine (shared host); non-Windows without a cert → keys stored plain, silently	Another site on the same server could unprotect the cookie keys	**FIX**
`AllowedHosts: "*"`	Host-header tricks (password-reset links, logs)	**FIX**
OpenAPI mapped only in Development (200 locally)	Must be 404 in production	**CHECK**
500s return ProblemDetails without exception text; CORS not enabled (cross-origin preflight → 405)	No stack traces; browsers on other sites can't call the API	**OK**
Secrets in user-secrets / env; `.env*` git-ignored; no keys found in tracked files		**OK**
NuGet `--vulnerable --include-transitive`: 0. npm audit (frontend): 0 critical, 0 high, 8 moderate (`minimatch`, dev only). No `.github/dependabot.yml`	New CVEs won't be noticed	**FIX**
Frontend sends no security headers (`next.config.ts` has no `headers()`); staging noindex not set	Teammate's area — block below	**FIX**

Fix: real client IP through Vercel (highest priority)

Next.js `rewrites()` forward the request but can't add headers, so `ClientIpMiddleware` never sees `X-Proxy-Secret` and ~~uses Vercel's IP for everyone. The frontend needs a `proxy.ts` (Next.js 16's name for middleware) — that file is the~~ teammate's. The API side should fail fast instead of silently falling back:

```
// frontend/src/proxy.ts (Next.js 16) — teammate. Forwards /api with the secret and the real IP.
import { NextResponse, type NextRequest } from "next/server";
export function proxy(request: NextRequest) {
  const url = new URL(request.nextUrl.pathname + request.nextUrl.search, process.env.API_URL);
  const headers = new Headers(request.headers);
  headers.set("X-Proxy-Secret", process.env.PROXY_SHARED_SECRET!);
  headers.set("X-Forwarded-For", request.headers.get("x-forwarded-for") ?? "");
  return NextResponse.rewrite(url, { request: { headers } });
}
export const config = { matcher: "/api/:path*" };
```

```
// Program.cs — after builder.Build(): refuse to run in production without the proxy secret
if (!app.Environment.IsDevelopment() && app.Configuration["Proxy:SharedSecret"] is not { Length: >= 32 })
    throw new InvalidOperationException("Proxy:SharedSecret (32+ chars) is required outside Development.");

// SecurityHeadersMiddleware — keep the API out of search engines
headers["X-Robots-Tag"] = "noindex, nofollow";
```

Recommended production config (API)

```
// appsettings.json
"Serilog": {
  "MinimumLevel": {
    "Default": "Information",
    "Override": {
      "Microsoft": "Warning",
      "Microsoft.EntityFrameworkCore": "Warning",
      "Microsoft.AspNetCore.Hosting.Diagnostics": "Warning",
      "System": "Warning"
    }
  }
},
"AllowedHosts": "api.zamzam-fashion.com"   // the real API host
```

```
<!-- web.config on MonsterASP -->
<system.webServer>
  <security><requestFiltering removeServerHeader="true" /></security>
  <httpProtocol><customHeaders><remove name="X-Powered-By" /></customHeaders></httpProtocol>
  <rewrite><rules>
    <rule name="https" stopProcessing="true"><match url="(.*)" />
      <conditions><add input="{HTTPS}" pattern="off" /></conditions>
      <action type="Redirect" url="https://{HTTP_HOST}/{R:1}" redirectType="Permanent" /></rule>
  </rules></rewrite>
</system.webServer>
```

Data-protection keys: create a self-signed .pfx once, store it base64 in MonsterASP's environment variables as `DataProtection__CertificateBase64` + `DataProtection__CertificatePassword`, and make the non-Windows fallback throw. Add `builder.Services.AddAuthorizationBuilder().SetFallbackPolicy(...)` requiring an authenticated user, so a future admin controller that forgets `[Authorize]` is closed by default (public controllers already carry `[AllowAnonymous]`).

Recommended security headers (frontend — teammate)

Start the CSP as `Report-Only`: a strict policy can break fonts, images from Supabase or analytics; switch to enforcing once the browser console shows no reports.

```
// frontend/next.config.ts — teammate
async headers() {
  return [{ source: "/(.*)", headers: [
    { key: "Strict-Transport-Security", value: "max-age=63072000; includeSubDomains; preload" },
    { key: "X-Content-Type-Options", value: "nosniff" },
    { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
    { key: "X-Frame-Options", value: "DENY" },
    { key: "Permissions-Policy", value: "camera=(), microphone=(), geolocation=()" },
    { key: "Content-Security-Policy-Report-Only", value:
      "default-src 'self'; img-src 'self' data: https://cngczppokulovwegzhdq.supabase.co; " +
      "script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; font-src 'self'; " +
      "connect-src 'self'; frame-ancestors 'none'" },
  ]}];
}
```

Forms and public endpoints

- **Already in place:** server-side validation filter, length caps in the DTO validators, idempotency key on checkout, per-IP limits (orders 10/10 min, tracking 30/min, estimate 60/min), tracking tokens stored as hashes and sent in a header.
- **Add before launch:** Cloudflare Turnstile on checkout and the tailoring request (verify the token in the API) — rate limits alone don't stop a botnet filling the order board.
- Keep customer email optional and never echo it in tracking responses (already masked: `+20111****678`).

Preview / staging hygiene

- Staging on the company domain: `X-Robots-Tag: noindex` and Vercel Deployment Protection on previews.
- Staging uses its own Supabase project or schema — never the production database.
- At launch: switch to the client's domain, add the canonical tag, remove noindex from production only.

3. SEO and sharing (quick check)

Item	Note	Status
Title, meta description, Open Graph / Twitter cards, canonical, robots, sitemap	Storefront not built yet	**CHECK**
API returns `X-Robots-Tag` ? No — but it is only reached through `/api` on the frontend domain	Add `X-Robots-Tag: noindex` to API responses so a stray link is never indexed	**FIX**
Accessibility basics (alt text, labels, heading order, skip link)	Day 5 — Lighthouse accessibility audit	**CHECK**

4. How to verify the CHECK items

Goal	Tool / command
Real Core Web Vitals (Day 5)	pagespeed.web.dev on the Vercel staging URL, mobile + desktop; WebPageTest.org for the waterfall
API latency from the production server	`curl -s -o NUL -w "%{time_total}" https://<api-host>/api/products?pageSize=12` ×30, compare with the 253 ms baseline
Server region / .NET 10 / IIS headers	`curl -sI https://<api-host>/health` → no `Server`, no `X-Powered-By`; MonsterASP panel for region + runtime
HTTP → HTTPS	`curl -sI http://<api-host>/health` → 301 to https
OpenAPI hidden in production	`curl -s -o NUL -w "%{http_code}" https://<api-host>/openapi/v1.json` → 404
Real client IP through Vercel	Call tracking 31× from two different networks via the staging site: only the noisy one gets 429; API log shows the real IP
Supabase pooler limit	Supabase dashboard → Database → Connection pooling (pool size); set `Max Pool Size` below it
Headers / TLS overall	securityheaders.com and developer.mozilla.org/observatory on the staging domain; SSL Labs on the API host
Accessibility	Lighthouse accessibility audit; axe DevTools
Dependencies	`dotnet list package --vulnerable --include-transitive`, `npm audit`; GitHub Dependabot alerts
Real capacity	`k6 run -e BASE_URL=https://<staging> zamzam-api-loadtest.js` (staging only, off-peak)

5. Priority action list

1. Send the proxy secret from the frontend (`src/proxy.ts`, teammate) and make the API refuse to start in Production without `Proxy:SharedSecret`. Confirm: two networks, only one gets 429.
2. Change the owner password (clears `mustChangePassword`). Confirm: `/api/auth/me` shows false.
3. Add a Serilog `MinimumLevel` block (Microsoft / EF at Warning). Confirm: one request = one log line.
4. Use the certificate for data-protection keys on MonsterASP (`DataProtection:CertificateBase64`) and throw if production has neither cert nor Windows. Confirm: login survives a redeploy.
5. Set `AllowedHosts`, `Database:MigrateOnStartup=false` and run migrations as a deploy step. Confirm: wrong Host header → 400.
6. Cache `GET /api/products` and `/{slug}` for 60 s and add response compression for JSON. Confirm: `Cache-Control` + `Content-Encoding: br` in `curl -sI`.

7. Decode photos scaled (SKCodec sample size to ~1600 px) and drop the second buffer. Confirm: upload a 40 MP photo, watch memory in Task Manager.
8. Add `.github/dependabot.yml` (nuget + npm + github-actions) and `Max Pool Size=10` in the connection string.

6. Capacity estimate (modelled)

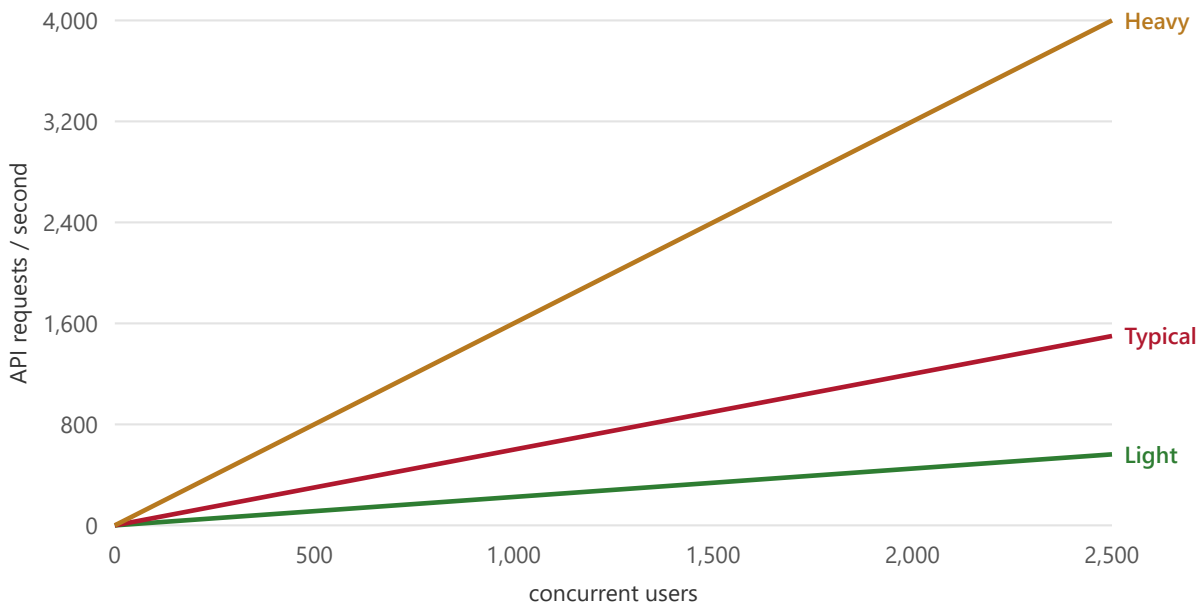
This section is a calculation, not a measurement.

No load was generated: the API isn't deployed and a load test against the shared Supabase dev database would be misleading. The model counts **API** requests only — images come from Supabase's CDN and pages from Vercel.

```
requests/sec = (concurrent users / read time in seconds) x API requests per page view
visits/sec   = concurrent users / read time (typical scenario, 5 s)
bandwidth    = visits/sec x first-visit MB x 8   (Mbps)
transfer/hr  = visits/sec x 3600 x first-visit MB / 1024   (GB)
```

Scenario	API requests / view	Read time	Reasoning
Light	1.8	8 s	Cached settings/zones, visitor skims a few pages
Typical	3	5 s	Product list or detail + settings + one cached lookup per page
Heavy	4.8	3 s	Cold cache, fast filtering and paging, estimate calls

API requests per second against concurrent users (calculation, not measured)



Concurrent users	Light (req/s)	Typical (req/s)	Heavy (req/s)
25	5.6	15.0	40.0
100	22.5	60.0	160.0
250	56.3	150.0	400.0
500	112.5	300.0	800.0
1,000	225.0	600.0	1,600.0
2,500	562.5	1,500.0	4,000.0

Bandwidth check (calculation)

Concurrent users	Visits / s	API Mbps (9.5 KB measured)	Full page Mbps (1.5 MB assumed)	Full page GB / hour
100	20.0	1.48	240.0	105.5

Concurrent users	Visits / s	API Mbps (9.5 KB measured)	Full page Mbps (1.5 MB assumed)	Full page GB / hour
500	100.0	7.41	1,200.0	527.3
1,000	200.0	14.83	2,400.0	1,054.7

How to read this

API bandwidth is tiny; the full-page traffic lands on Vercel and Supabase's CDN, so check both plans' monthly transfer quotas before any campaign. The real limit is the database: with `Max Pool Size=10` and ~253 ms per uncached product list, one instance serves about **40 uncached list requests per second** (calculation: 10 / 0.253 s) — roughly the Typical line at 66 concurrent users if nothing were cached. That's why the 60-second product cache and an EU server matter more than anything else here: with caching, most of those requests never reach the API. Checkout and tailoring requests are the expensive dynamic calls (transaction + retries; photo decode) — they are rate-limited and should be tested separately at low rate.

The k6 script `zamzam-api-loadtest.js` (delivered with this report) ramps browsing to 100 users on the public GET endpoints and runs a separate 2-per-second scenario on `POST /api/tailoring/estimate`, which computes a price and writes nothing — it never places orders or sends email. Run it only against your own staging deployment, outside peak hours, and only after the proxy-secret fix (otherwise k6 itself trips the per-IP limits).

Assumptions: 3 API calls per page view (Typical), 0.6× / 1.6× for Light / Heavy; read times 8 / 5 / 3 s; API JSON first visit 9.5 KB measured (categories 586 B, product list 3,792 B, product 1,720 B, settings 533 B, zones 3,085 B); full first page 1.5 MB assumed because the storefront isn't built; DB time per list from the local p50. **Limitations:** findings come from the source code and the API running locally at the time of review; no intrusive or active security testing was performed, and nothing was measured on the production host. This report can be updated with the PageSpeed run, `curl -sI` output from MonsterASP and the k6 summary.